This works fine, returning a new `Person` object. But later, someone might subclass our class:

```
class Employee < Person
  # ..
end

dave = Employee.for_dave  # returns a Person
```

The `for_dave` method was hardwired to return a `Person` object, so that's what's returned by `Employee.for_dave`. Using `self.new` instead returns a new object of the receiver's class, `Employee`.

lambda

The `lambda` operator converts a block into an object of type `Proc`. An alternative syntax, introduced in Ruby 1.9, is `->`. As a matter of style, the Rails team prefers the latter syntax. You can see example usages of this operator in *Scopes*.

require File.expand_path("../../config/environment", __FILE__)

Ruby's `require` method loads an external source file into our application. This is used to include library code and classes that our application relies on. In normal use, Ruby finds these files by searching in a list of directories, the `LOAD_PATH`.

Sometimes we need to be specific about which file to include. We can do that by giving `require` a full filesystem path. The problem is, we don't know what that path will be—our users could install our code anywhere.

Wherever our application ends up getting installed, the relative path between the file doing the requiring and the target file will be the same. Knowing this, we can construct the absolute path to the target by using the `File.expand_path` method, passing in the relative path to the target file, and passing the absolute path to the file doing the requiring (available in the special `__FILE__` variable).